JavaGenerator.py 解説

JavaParser が出力した JSON から、nfc を TeaVM で Wasm 化するための Java のエントポイント（公開窓口）・ブリッジと、JS のラッパーを jinja2 で生成するプログラムを、1 行ずつ説明します。

全体の中での位置

① 解析（Java / JavaParser）
`./gradlew :app:parseJava --args="… --json build/nfc_api.json"`
nfc のクラス・メソッド・引数・戻り値を JSON にする

→

② 生成（Python / jinja2）← この資料
`python3 app/src/main/Python/JavaGenerator.py`
JSON を読み、ブリッジ・NfcWasm・JS ラッパーのソースを書き出す

→

③ Wasm 化（Gradle / TeaVM）
`./gradlew :app:build`
mainClass を `org.example.generated.NfcWasm` にすると、生成したエントポイントが Wasm に入る

ファイル構成

ファイル	役割
`JavaGenerator.py`	JSON を読み、何をどう生成するかを決めて、テンプレートに渡す
`templates/bridge.java.j2`	→ `EigWasm.java` など（パッケージごとのブリッジ）
`templates/nfc_wasm.java.j2`	→ `NfcWasm.java`（@JSExport の公開窓口）
`templates/matrices.java.j2`	→ `Matrices.java`（配列の変換。手書き版のコピー）
`templates/nfc.js.j2`	→ `webapp/nfc.generated.js`（JS ラッパー）
`templates/report.md.j2`	→ `build/codegen_report.md`（生成結果の一覧）

関数の呼ばれ方

```
main()                              ⑫ 引数を受け取り、JSON を読む
├─ build_bridges()                  ⑩ 全体を組み立てる
│    ├─ select_entry_classes()      ⑥ 手順A・B 入口クラスを選ぶ
│    ├─ build_method()              ⑧ 手順C メソッド 1 つ分を変換
│    │    ├─ find_getters()         ⑦ 手順D 箱の getter を探す
│    │    └─ base_export_name()     ⑨ 公開名を作る
│    └─ resolve_name_collisions()   ⑨ 名前の重複を解決
└─ render_all()                     ⑪ テンプレートで書き出す
```

役割の分担：**Python** が「何を、どう変換するか」をすべて決め、**テンプレート**は受け取った値を並べるだけにしています。

具体例：DoubleRealSchurDecomposer.decompose が生成されるまで

入力：JSON（抜粋）

```
{ "name": "DoubleRealSchurDecomposer", "kind": "class",
  "constructors": [ { "parameters": [], "implicit": true } ],
  "methods": [ { "name": "decompose", "static": false,
      "returnType": "SchurDecompositionDoubleRealElements",
      "parameters": [ { "type": "double[][]", "name": "a" } ] } ] }

{ "name": "SchurDecompositionDoubleRealElements",      ← 箱
  "methods": [ { "name": "getU", "returnType": "double[][]", "paramete
rs": [] },
               { "name": "getT", "returnType": "double[][]", "paramete
rs": [] } ] }
```

build_method が作る辞書（テンプレートに渡すデータ）

```
{ "source": "DoubleRealSchurDecomposer.decompose",
  "export_name": "realSchurDecompose",
  "original_type": "SchurDecompositionDoubleRealElements",
  "call": "new DoubleRealSchurDecomposer().decompose(Matrices.toMatrix
(a, aRows, aColumns))",
  "java_params": ["double[] a", "int aRows", "int aColumns"],
  "java_args": ["a", "aRows", "aColumns"],
  "js_args": ["aF.values", "aF.rows", "aF.cols"],
  "result": { "java": "double[]", "js": "object",
    "expr": "Matrices.pack(result.getU(), result.getT())",
    "fields": [ { "key": "U", … }, { "key": "T", … } ] },
  "imports": { "org.mklab.nfc.eig.DoubleRealSchurDecomposer",
               "org.mklab.nfc.eig.SchurDecompositionDoubleRealElement
s" } }
```

→ EigWasm.java（bridge.java.j2）

```
static double[] realSchurDecompose(double[]
a, int aRows, int aColumns) {
    SchurDecompositionDoubleRealElements re
sult =
        new DoubleRealSchurDecomposer().dec
ompose(Matrices.toMatrix(a, aRows, aColumn
s));
    return Matrices.pack(result.getU(), res
ult.getT());
}
```

→ NfcWasm.java（nfc_wasm.java.j2）

```
@JSExport
public static double[] realSchurDecompose(d
ouble[] a, int aRows, int aColumns) {
    return EigWasm.realSchurDecompose(a, aR
ows, aColumns);
}
```

→ nfc.generated.js（nfc.js.j2）

```
realSchurDecompose(a) {
  const aF = flatten(a);
  return named(unpack(ex.realSchurDecompose
(aF.values, aF.rows, aF.cols)),
               { U: "matrix", T: "matrix"
});
}
// 使う側: const { U, T } = nfc.realSchurDec
ompose([[1,2],[3,4]]);
```

先に知っておくとよい書き方

Python

辞書 {"k": v}	キーで値を引ける表。JSON は読み込むと辞書とリストになる。`c["name"]` で取り出す。
リスト内包表記	`[式 for x in リスト if 条件]` で新しいリストを 1 行で作る。`{k: v for …}` は辞書版。
f"…{式}…"	f 文字列。`{ }` の中の式の値が埋め込まれる。
"…{n}…".format(n=…)	文字列の `{n}` を後から置き換える。変換表の `{n}` に引数名を入れるのに使う。
any() / next()	`any` は「1 つでも真なら真」。`next` は条件に合う最初の 1 つを取り出す。
set と |=	重複しない集まり。`a |= b` で b の要素を a に足す（import の重複を防ぐ）。
setdefault	キーが無ければ既定値を入れ、あればその値を返す。グループ分けに便利。
raise / try-except	例外を投げる／捕まえる。ここでは「対象外」を理由付きで伝えるのに使う（Skip）。
Path と /	`Path("a") / "b"` でパスをつなぐ。`.parent` で親フォルダ。

jinja2（テンプレート）

{{ 式 }}	値を出力する。`{{ m.export_name }}` のように、辞書の中身は `.` で取り出せる。
{% for %} / {% if %}	繰り返しと条件分岐。`{% endfor %}` / `{% endif %}` で閉じる。
{% for x in xs if 条件 %}	条件に合う要素だけを繰り返す。
{# … #}	コメント。出力されない。
| join(", ")	フィルタ。リストを「, 」でつなげて 1 つの文字列にする。
| map(attribute="name")	リストの各要素から name だけを取り出す。
| length / | sum	個数を数える／足し合わせる。
loop.last	for の最後の 1 回なら真。区切りの `,` を最後だけ付けないのに使う。
{% set x = … %} / ~	テンプレート内で変数を作る。`~` は文字列の連結。
trim_blocks 等	`{% %}` の行が余計な空行・空白として出力されないようにする設定（JavaGenerator.py の 305〜306 行目）。

設計のポイント

- **変換表を辞書にする**：ガイドの「JS に渡せる型」の表を `PARAM_RULES` などの辞書にしたので、if 文が増えない。新しい型への対応は表に 1 行足すだけ。
- **対象外は Skip 例外で記録**：生成できないメソッドは理由付きで記録し、レポートに出す。黙って消えるものが無い。
- **型を明示して import**：生成コードは `var` を使わず、`SchurDecompositionDoubleRealElements result = …` と書く（読みやすく、手書き版と比べやすい）。

- **手書き版には触れない**：出力は `org.example.generated` パッケージと `nfc.generated.js`。手書き版と結果が一致することを確認済み（並び順と空の P 以外は同一）。
- **null は空の行列**：置換なしの QR／LU 分解では `getP()` が null を返すので、生成版の Matrices では null を 0 x 0 の行列として扱う（JS では `P: []`）。
- **matrices.java.j2** は手書きの Matrices.java のコピー（package と null の扱いだけ違う）なので、この資料では 1 行ずつの説明を省略しています。

JavaGenerator.py の 1 行ずつの説明

① ファイルの説明（docstring）

このファイルが何を生成するか、ガイドの手順とどの関数が対応するか、実行方法を書いた説明文です。

行	コード	説明
1	"""	「"""」で囲んだ部分はドキュメント文字列（docstring）。ファイルの先頭に書くと、このファイル全体の説明になる。実行には影響しない。
2	JavaParser が出力した API の JSON（nfc_api.json）から、TeaVM 用のブリッジコードを jinja2 で生成する。	このプログラムの目的：JSON を読み、jinja2 で TeaVM 用の Java（と JS）のコードを生成すること。
3		
4	生成するもの:	ここから、生成するファイルの一覧。
5	- org/example/generated/〇〇Wasm.java … nfc のパッケージごとのブリッジ（型の変換 → 呼び出し → pack）	nfc のパッケージ（eig, leq など）ごとに 1 つ作るブリッジクラス。型の変換と呼び出しを担当する。
6	- org/example/generated/NfcWasm.java … @JSExport を付けた公開窓口	@JSExport を付けた公開窓口（エンドポイント）。TeaVM の mainClass に指定するクラス。
7	- org/example/generated/Matrices.java … 配列の変換（手書き版とほぼ同じ。null を空の行列として扱う）	配列の変換をする補助クラス。手書き版とほぼ同じだが、null を空の行列として扱う点が違う。
8	- webapp/nfc.generated.js … JS から 2 次元配列のまま呼べるラッパー	JS から 2 次元配列のまま呼び出せるようにするラッパー。
9	- build/codegen_report.md … 生成したメソッドと、対象外にしたメソッドの一覧	何を生成し、何を対象外にしたかをまとめたレポート。
10		
11	手順は WASM_EXPORT_GUIDE.md に対応している:	WASM_EXPORT_GUIDE.md の手順と、このファイルの関数の対応表。
12	手順A・B … select_entry_classes() 入口クラスを選ぶ	手順 A・B（入口クラスを選ぶ）は select_entry_classes 関数。
13	手順C … build_method() メソッドの static・引数・戻り値を見て変換方法を決める	手順 C（static か・引数・戻り値を見る）は build_method 関数。
14	手順D … find_getters() 戻り値が「箱」なら getter を探す	手順 D（戻り値が箱なら getter を探す）は find_getters 関数。
15		
16	使い方（TeaVM フォルダで）:	実行方法の説明。
17	./gradlew :app:parseJava --args="app/src/main/java/org/mklab/nfc --json build/nfc_api.json"	① 先に Gradle の parseJava タスクで JSON を作る。

行	コード	説明
18	`  python3 app/src/main/Python/JavaGenerator.py`	② その後このファイルを実行する。
19	`"""`	docstring の終わり。
20		

② import

使うライブラリを読み込みます。jinja2 以外は Python に標準で入っています。

行	コード	説明
21	`import argparse`	argparse：コマンドライン引数（--json など）を受け取るための標準ライブラリ。
22	`import json`	json：JSON ファイルを読み込んで、Python の辞書やリストに変換する標準ライブラリ。
23	`import re`	re：正規表現（文字列のパターン検索・置換）の標準ライブラリ。公開名を作るときに使う。
24	`from pathlib import Path`	Path：ファイルの場所を扱うクラス。「/」演算子でフォルダ名をつなげられる。
25		
26	`from jinja2 import Environment, FileSystemLoader`	jinja2 から、テンプレートを扱う Environment と、フォルダからテンプレートを読む FileSystemLoader を読み込む。
27		

③ 設定：入出力の場所と、入口の選び方のルール

後から変えそうな値を、ファイルの先頭に定数としてまとめています。大文字の名前は「定数」という Python の慣習です。

行	コード	説明
28	`# ----------------------------------------------------------------------------`	区切り線のコメント（ここから「設定」のまとまり）。
29	`# 設定`	このまとまりでは、入出力の場所や、選び方のルールを定数として定義する。
30	`# ----------------------------------------------------------------------------`	区切り線。
31		
32	`# このファイルは TeaVM/app/src/main/Python/ にあるので、4 つ上が TeaVM フォルダ`	下の 2 行の説明コメント。
33	`BASE_DIR = Path(__file__).resolve().parent`	__file__ はこのファイル自身のパス。resolve() で絶対パスにし、.parent でそのフォルダ（Python/）を得る。

行	コード	説明
34	`PROJECT_ROOT = BASE_DIR.parents[3]`	parents[3] は 4 つ上のフォルダ。Python → main → src → app の上なので TeaVM フォルダになる。
35		
36	`DEFAULT_JSON = PROJECT_ROOT / "build" / "nfc_api.json"`	入力 JSON の既定の場所：TeaVM/build/nfc_api.json。Path 同士を「/」でつなげている。
37	`JAVA_PACKAGE = "org.example.generated"`	生成する Java のパッケージ名。手書き版（org.example）とぶつからないように別にしている。
38	`JAVA_OUT_DIR = PROJECT_ROOT / "app" / "src" / "main" / "java" / Path(*JAVA_PACKAGE.split("."))`	Java の出力先フォルダ。split(".") で ["org","example","generated"] にし、Path(*…) で org/example/generated にする。
39	`JS_OUT_FILE = PROJECT_ROOT / "app" / "src" / "main" / "webapp" / "nfc.generated.js"`	JS ラッパーの出力先。
40	`REPORT_FILE = PROJECT_ROOT / "build" / "codegen_report.md"`	レポートの出力先。build/ に置くので、作り直してよい生成物の扱い。
41		
42	`# 手順A：入口クラスとみなすクラス名の末尾`	手順 A のルールの説明。
43	`ENTRY_SUFFIXES = ("Solver", "Decomposer", "Analyzer", "ExponentialMatrix")`	入口クラスとみなすクラス名の末尾。タプル（変更しない並び）にしているのは、endswith にそのまま渡せるから。
44	`# 手順B：倍精度の実数版・複素数版だけを対象にする（JS 側の名前の先頭に付ける語）`	手順 B のルールの説明。
45	`ENTRY_PREFIXES = {"DoubleReal": "real", "DoubleComplex": "complex"}`	対象にするクラス名の先頭と、公開名の先頭に付ける語の対応。DoubleReal… → real…、DoubleComplex… → complex…。
46	`# 公開名を短くするための言い換え（クラス名の中心部分 → 公開名で使う語）`	下の辞書の説明。
47	`CORE_ABBREVIATIONS = {"SingularValue": "Svd"}`	公開名が長くなりすぎないための言い換え。SingularValue → Svd（例：realSvdRank）。
48		

④ 型の変換表（PARAM_RULES / RETURN_RULES / GETTER_RULES）

WASM_EXPORT_GUIDE.md の「JS に渡せる型・渡せない型」の表を、そのまま辞書にしたものです。型ごとの if 文を書き並べる代わりに、この表を引くだけで変換方法が決まります。新しい型に対応したいときは、ここに 1 行足します。

行	コード	説明
49	`# ----------------------------------------------------------------------------`	区切り線（ここから「型の変換表」）。

行	コード	説明
50	`# 型の変換表`	Java の型ごとに、どう受け渡すかを辞書（表）にまとめる。条件分岐を if で書き並べる代わりに、表を引くだけで済むようにしている。
51	`# ----------------------------------------------------------------------`	区切り線。
52		
53	`# 引数：Java の型 → ブリッジでの受け取り方`	引数用の変換表の説明。
54	`#   java : ブリッジ／NfcWasm の引数（{n} は元の引数名）`	java：ブリッジと NfcWasm での引数の書き方。{n} は後で元の引数名に置き換える。
55	`#   arg  : 元のメソッドに渡す式`	arg：元の nfc のメソッドに渡すときの式。
56	`#   js   : JS 側の扱い（"matrix" なら 2 次元配列を flatten して渡す）`	js：JS 側でどう扱うか。matrix なら 2 次元配列を flatten して渡す。
57	`PARAM_RULES = {`	引数の変換表の始まり。キーが Java の型名（JSON の "type" と同じ文字列）。
58	`    "double": {"java": ["double {n}"], "arg": "{n}", "js": "plain"},`	double はそのまま受け取り、そのまま渡せる。
59	`    "int": {"java": ["int {n}"], "arg": "{n}", "js": "plain"},`	int もそのまま。
60	`    "boolean": {"java": ["boolean {n}"], "arg": "{n}", "js": "plain"},`	boolean もそのまま。
61	`    "double[]": {"java": ["double[] {n}"], "arg": "{n}", "js": "plain"},`	double[]（1 次元配列）も JS の配列からそのまま渡せる。
62	`    "double[][]": {`	double[][]（2 次元配列）は JS から直接渡せないので変換が必要。
63	`        "java": ["double[] {n}", "int {n}Rows", "int {n}Columns"],`	「1 次元配列 a + 行数 aRows + 列数 aColumns」の 3 つの引数に分けて受け取る。
64	`        "arg": "Matrices.toMatrix({n}, {n}Rows, {n}Columns)",`	元のメソッドには、Matrices.toMatrix で 2 次元配列に戻してから渡す。
65	`        "js": "matrix",`	JS 側では flatten して 3 つの値にして渡す。
66	`    },`	double[][] の設定の終わり。
67	`    "DoubleMatrix": {`	DoubleMatrix（nfc の行列クラス）も同じく 3 つの引数で受け取る。
68	`        "java": ["double[] {n}", "int {n}Rows", "int {n}Columns"],`	受け取り方は double[][] と同じ。
69	`        "arg": "new DoubleMatrix(Matrices.toMatrix({n}, {n}Rows, {n}Columns))",`	2 次元配列に戻した後、new DoubleMatrix(…) でオブジェクトにしてから渡す。
70	`        "js": "matrix",`	JS 側の扱いも同じ。
71	`    },`	DoubleMatrix の設定の終わり。
72	`}`	引数の変換表の終わり。
73		

行	コード	説明
74	`# 戻り値：Java の型 → JS に返す形`	戻り値用の変換表の説明。
75	`#   java : ブリッジ／NfcWasm の戻り値の型`	java：ブリッジと NfcWasm の戻り値の型。
76	`#   expr : 変数 result を JS に返す式（None なら何も返さない）`	expr：nfc から受け取った値（変数 result）を JS に返す形に変える式。None なら何も返さない（void）。
77	`#   js   : JS 側で unpack した後の形`	js：JS 側で unpack した後の形。
78	`RETURN_RULES = {`	戻り値の変換表の始まり。
79	`    "void": {"java": "void", "expr": None, "js": "none"},`	void は何も返さない。
80	`    "double": {"java": "double", "expr": "result", "js": "plain"},`	double はそのまま返せる。
81	`    "int": {"java": "int", "expr": "result", "js": "plain"},`	int もそのまま。
82	`    "boolean": {"java": "boolean", "expr": "result", "js": "plain"},`	boolean もそのまま。
83	`    "double[]": {"java": "double[]", "expr": "Matrices.pack(Matrices.rowVector(result))", "js": "vector"},`	double[] は 1 x n の行列として pack し、JS では 1 次元配列（vector）に戻す。
84	`    "double[][]": {"java": "double[]", "expr": "Matrices.pack(result)", "js": "matrix"},`	double[][] は pack して 1 本の double[] にする。JS では行列 1 つに戻す。
85	`    "double[][][]": {"java": "double[]", "expr": "Matrices.pack(result)", "js": "matrices"},`	double[][][]（行列の配列）は pack にそのまま渡せる（pack は可変長引数なので配列も受け取れる）。JS では行列の配列に戻す。
86	`    "int[][]": {"java": "double[]", "expr": "Matrices.pack(Matrices.toDouble(result))", "js": "matrix"},`	int[][]（置換行列など）は toDouble で double に変換してから pack する。
87	`    "DoubleMatrix": {"java": "double[]", "expr": "Matrices.pack(result.getDoubleElements())", "js": "matrix"},`	DoubleMatrix は getDoubleElements() で 2 次元配列を取り出してから pack する。
88	`}`	戻り値の変換表の終わり。
89		
90	`# 手順D：箱の getter の戻り値の型 → pack に渡せる double[][] にする式（{g} は getter の呼び出し）`	手順 D 用の表の説明。箱の getter が返す型を、pack に渡せる double[][] にする方法。
91	`GETTER_RULES = {`	getter 用の変換表の始まり。
92	`    "double[][]": {"expr": "{g}", "js": "matrix"},`	double[][] はそのまま pack に渡せる。
93	`    "double[]": {"expr": "Matrices.rowVector({g})", "js": "vector"},`	double[]（固有値の実部など）は rowVector で 1 x n 行列にする。JS では 1 次元配列に戻す。
94	`    "int[][]": {"expr": "Matrices.toDouble({g})", "js": "matrix"},`	int[][] は toDouble で double に変換する。

行	コード	説明
95	`}`	getter 用の変換表の終わり。
96		
97		

⑤ Skip 例外

「このメソッドは生成しない」ことを、理由付きで呼び出し元に知らせるための仕組みです。

行	コード	説明
98	`class Skip(Exception):`	Skip という独自の例外クラスを定義。Exception を継承している。
99	`    """このメソッドは生成の対象外（理由をメッセージに入れる）。"""`	「このメソッドは生成しない」ことを知らせる。深い場所から理由付きで一気に抜け出し、呼び出し元でまとめて記録するために使う。
100		
101		

⑥ select_entry_classes：入口クラスを選ぶ（手順 A・B）

JSON の全クラスから、4 つの条件をすべて満たすクラスだけを残します。

行	コード	説明
102	`# ----------------------------------------------------------------------------`	区切り線（ここから手順 A・B）。
103	`# 手順A・B：入口クラスを選ぶ`	入口クラスを選ぶ処理のまとまり。
104	`# ----------------------------------------------------------------------------`	区切り線。
105		
106	`def select_entry_classes(classes):`	入口クラスを選ぶ関数。classes は JSON の "classes"（全クラスのリスト）。
107	`    """クラス名と形から、JS に公開する入口クラスだけを選ぶ。"""`	関数の説明（docstring）。
108	`    entries = []`	選ばれたクラスを入れる空のリスト。
109	`    for c in classes:`	全クラスを 1 つずつ c として取り出して調べる。c は JSON のクラス 1 つ分の辞書。
110	`        is_entry_name = c["name"].endswith(ENTRY_SUFFIXES)`	条件①：名前が Solver / Decomposer / Analyzer / ExponentialMatrix のどれかで終わるか（手順 A）。endswith にタプルを渡すと「どれか」で判定できる。

行	コード	説明
111	`        is_double = c["name"].startswith(tuple(ENTRY_PREFIXES))`	条件②：名前が DoubleReal か DoubleComplex で始まるか（手順 B）。tuple(辞書) で辞書のキーだけの並びになる。
112	`        is_concrete_class = c["kind"] == "class" and "abstract" not in c["modifiers"]`	条件③：普通のクラスか（interface ではなく、abstract でもない）。new したり呼び出したりできるものだけにする。
113	`        is_not_generic = not c["typeParameters"]`	条件④：型パラメータ（<RS, RM, …>）が無いか。空のリストは偽なので、not で「空なら真」になる。
114	`        if is_entry_name and is_double and is_concrete_class and is_not_generic:`	4 つの条件がすべて真なら…
115	`            entries.append(c)`	選ばれたリストに追加する。
116	`    return entries`	選ばれた入口クラスのリストを返す。
117		
118		

⑦ find_getters：箱の getter を探す（手順 D）

戻り値が「箱」だったとき、中身を取り出すための getter を集めます。親クラスにある getter もたどります。

行	コード	説明
119	`# ----------------------------------------------------------------------------`	区切り線（ここから手順 D）。
120	`# 手順D：戻り値が「箱」なら getter を探す`	戻り値が箱のとき、中身を取り出す getter を探すまとまり。
121	`# ----------------------------------------------------------------------------`	区切り線。
122		
123	`def find_getters(box, class_by_name):`	getter を探す関数。box は箱のクラス、class_by_name は「クラス名 → クラス」の辞書。
124	`    """箱のクラス（親クラスも含む）から、get で始まり引数 0 個で、変換できる型を返す getter を集める。"""`	関数の説明。親クラスもたどること、getter の条件を書いている。
125	`    getters = []`	見つかった getter を入れるリスト。
126	`    current = box`	今調べているクラス。最初は箱のクラス自身。
127	`    while current is not None:`	親クラスが無くなる（None になる）まで繰り返す。
128	`        for m in current["methods"]:`	今のクラスのメソッドを 1 つずつ調べる。

行	コード	説明
129	`            if (m["name"].startswith("get") and not m["parameters"] and not m["static"]`	getter の条件：名前が get で始まり、引数が 0 個で、static ではなく…
130	`                    and m["returnType"] in GETTER_RULES):`	…戻り値の型が getter 用の変換表にあるもの（変換できる型）。containP（boolean）などはここで除かれる。
131	`                getters.append(m)`	条件を満たしたら getter として追加する。
132	`        # extends が JSON にあるクラスなら、親クラスもたどる（List<String> のような <...> は外す）`	下の 2 行の説明。
133	`        parent_name = current["extends"][0].split("<")[0] if current["extends"] else None`	extends があれば先頭の型名を取り出す。「Foo<Bar>」なら split("<")[0] で「Foo」だけにする。無ければ None。
134	`        current = class_by_name.get(parent_name)`	親クラスを辞書から探す。見つからなければ get は None を返すので、ループが終わる。
135	`    return getters`	集めた getter のリストを返す。
136		
137		

⑧ build_method：メソッド 1 つ分の変換方法を決める（手順 C）

この生成器の中心です。呼び出し方（static か new か）→ 引数 → 戻り値 の順に変換表を引き、テンプレートに渡す辞書を作ります。生成できない場合は Skip を投げます。

行	コード	説明
138	`# ----------------------------------------------------------------------------`	区切り線（ここから手順 C）。
139	`# 手順C：メソッド 1 つ分の変換方法を決める`	メソッド 1 つ分の変換方法を決めるまとまり。この関数が生成器の中心。
140	`# ----------------------------------------------------------------------------`	区切り線。
141		
142	`def build_method(entry, method, class_by_name):`	メソッド 1 つを、テンプレートに渡す辞書に変換する関数。entry は入口クラス、method はそのメソッド。
143	`    """入口クラスのメソッド 1 つを、テンプレートに渡す辞書に変換する。対象外なら Skip を投げる。"""`	関数の説明。生成できないときは Skip 例外を投げる。
144	`    if any(a.startswith("@Deprecated") for a in method["annotations"]):`	アノテーションに @Deprecated（非推奨）が付いているか調べる。any は「どれか 1 つでも真なら真」。

行	コード	説明
145	`        raise Skip("@Deprecated")`	非推奨なら Skip を投げて対象外にする。
146	`    if method["typeParameters"]:`	メソッド自身が型パラメータ（<T> など）を持っていたら…
147	`        raise Skip("ジェネリックメソッド")`	JS からは型を指定できないので対象外。
148		
149	`    # 呼び出し方：static なら「クラス名.メソッド」、そうでなければ new してから呼ぶ`	呼び出し方の決め方の説明。
150	`    if method["static"]:`	static メソッドなら…
151	`        receiver = entry["name"]`	「クラス名.メソッド(…)」と呼ぶので、呼び出し先（receiver）はクラス名。
152	`    else:`	static でなければ…
153	`        if method["returnType"] == "void":`	戻り値が void なら…（例：setMaxIteration）
154	`            raise Skip("毎回 new したインスタンスに対する void メソッドなので、呼んでも効果がない")`	毎回 new した使い捨てのインスタンスの設定を変えても意味がないので対象外。
155	`        if not any(not k["parameters"] for k in entry["constructors"]):`	引数 0 個のコンストラクタが 1 つも無ければ…（new できない）
156	`            raise Skip("引数なしの public コンストラクタがない")`	対象外。
157	`        receiver = f"new {entry['name']}()"`	「new クラス名()」を呼び出し先にする。f"…" は f 文字列で、{} の中の式が埋め込まれる。
158		
159	`    imports = {entry["qualifiedName"]}`	このメソッドを呼ぶために import が必要なクラスの集合（set）。まず入口クラス自身を入れる。set なので同じものは 1 つにまとまる。
160		
161	`    # 引数`	ここから引数の処理。
162	`    params = []`	変換後の引数の情報を入れるリスト。
163	`    for p in method["parameters"]:`	元のメソッドの引数を 1 つずつ p として取り出す。
164	`        rule = PARAM_RULES.get(p["type"])`	引数の型で変換表を引く。表に無い型なら get は None を返す。
165	`        if rule is None or p["varArgs"]:`	表に無い型、または可変長引数（…）なら…
166	`            raise Skip(f"引数の型 {p['type']}{'...' if p['varArgs'] else ''} を JS から渡せない")`	理由付きで対象外にする（例：「引数の型 CS[][] を JS から渡せない」）。
167	`        if p["type"] in class_by_name:`	引数の型が nfc のクラス（DoubleMatrix など）なら…
168	`            imports.add(class_by_name[p["type"]]["qualifiedName"])`	その完全修飾名を import に加える。

行	コード	説明
169	`        params.append({`	変換後の引数情報を辞書にして追加する。
170	`            "name": p["name"],`	元の引数名（例：a）。
171	`            "java": [s.format(n=p["name"]) for s in rule["java"]],`	ブリッジでの引数の書き方。{n} を引数名に置き換える（例：["double[] a", "int aRows", "int aColumns"]）。
172	`            "arg": rule["arg"].format(n=p["name"]),`	元のメソッドに渡す式（例：Matrices.toMatrix(a, aRows, aColumns)）。
173	`            "js": rule["js"],`	JS 側の扱い（plain か matrix）。
174	`        })`	辞書の終わり。
175		
176	`    # 戻り値`	ここから戻り値の処理。
177	`    return_type = method["returnType"]`	戻り値の型を変数に入れておく。
178	`    if return_type in RETURN_RULES:`	戻り値の型が戻り値の変換表にあるなら（double[][] など、箱ではない型）…
179	`        rule = RETURN_RULES[return_type]`	その型の変換方法を取り出す。
180	`        result = {"java": rule["java"], "expr": rule["expr"], "js": rule["js"], "fields": []}`	テンプレートに渡す戻り値の情報。箱ではないので fields（取り出す中身の一覧）は空。
181	`        if return_type in class_by_name:`	戻り値が nfc のクラス（DoubleMatrix）なら…
182	`            imports.add(class_by_name[return_type]["qualifiedName"])`	import に加える。
183	`    elif return_type in class_by_name:`	表には無いが、JSON の classes にある型なら、それは「箱」（手順 D）。
184	`        # 手順D：箱 → getter で取り出して 1 本の配列に詰める`	箱のときの処理の説明。
185	`        box = class_by_name[return_type]`	箱のクラスの情報を辞書から取り出す。
186	`        getters = find_getters(box, class_by_name)`	getter を探す（find_getters）。
187	`        if not getters:`	取り出せる getter が 1 つも無ければ…
188	`            raise Skip(f"戻り値 {return_type} に取り出せる getter がない")`	対象外。
189	`        imports.add(box["qualifiedName"])`	箱のクラスを import に加える（ブリッジで型名を明示して書くため）。
190	`        fields = [{`	getter ごとに、取り出し方をまとめたリストを作る（リスト内包表記）。
191	`            "key": g["name"][len("get"):],`	JS 側のキー名。getter 名から先頭の get を取る（getU → U）。len("get") は 3。
192	`            "expr": GETTER_RULES[g["returnType"]]["expr"].format(g=f"result.{g['name']}()"),`	pack に渡す式。例：result.getU()。double[] なら Matrices.rowVector(result.getReValue()) になる。

行	コード	説明
193	`            "js": GETTER_RULES[g["returnType"]]["js"],`	JS 側でその値を行列として戻すか、1 次元配列として戻すか。
194	`        } for g in getters]`	全 getter について繰り返す（内包表記の for 部分）。
195	`        result = {`	テンプレートに渡す戻り値の情報。
196	`            "java": "double[]",`	ブリッジは常に pack した double[] を返す。
197	`            "expr": "Matrices.pack(" + ", ".join(f["expr"] for f in fields) + ")",`	全 getter の式を「, 」でつなげて pack に渡す式を作る。例：Matrices.pack(result.getU(), result.getT())。
198	`            "js": "object",`	JS 側では名前付きのオブジェクト（{ U, T }）として返す。
199	`            "fields": fields,`	取り出す中身の一覧。JS 側で名前を付けるのに使う。
200	`        }`	辞書の終わり。
201	`    else:`	表にも classes にも無い型（Object[] など）なら…
202	`        raise Skip(f"戻り値の型 {return_type} を JS に返せない")`	対象外。
203		
204	`    # ブリッジ／NfcWasm の引数（double[][] は 3 つに分かれるので、ここで 1 列に並べておく）`	下の行の説明。
205	`    java_params = [decl for p in params for decl in p["java"]]`	全引数の java（宣言）を 1 列に並べる。double[][] の引数は 3 つに分かれているので、ここで平らにしておく（二重 for の内包表記）。
206		
207	`    return {`	ここから、テンプレートに渡す辞書を返す。
208	`        "source": f"{entry['name']}.{method['name']}",`	元のメソッド名（例：DoubleRealSchurDecomposer.decompose）。コメントやレポートに使う。
209	`        "export_name": base_export_name(entry["name"], method["name"]),`	JS から呼ぶときの名前（例：realSchurDecompose）。base_export_name で作る。
210	`        "original_type": return_type,`	元の戻り値の型。ブリッジで「型 result = …」と明示して書くのに使う。
211	`        "call": f"{receiver}.{method['name']}(" + ", ".join(p["arg"] for p in params) + ")",`	nfc を呼び出す式全体。例：new DoubleRealSchurDecomposer().decompose(Matrices.toMatrix(a, aRows, aColumns))。
212	`        "throws": method["throws"],`	throws（投げる可能性のある例外）。あればブリッジで try-catch を書く。
213	`        "params": params,`	引数の情報（JS 側で flatten が必要かの判定に使う）。

行	コード	説明
214	`        "java_params": java_params,                                  # 例: ["double[] a", "int aRows", "int aColumns"]`	ブリッジ／NfcWasm の引数宣言の並び。
215	`        "java_args": [decl.split()[-1] for decl in java_params], # 例: ["a", "aRows", "aColumns"]`	NfcWasm からブリッジを呼ぶときの引数名の並び。宣言「double[] a」の最後の単語「a」を取り出している。
216	`        "js_args": [arg for p in params for arg in                   # 例: ["aF.values", "aF.rows", "aF.cols"]`	JS から Wasm を呼ぶときの引数の並び。
217	`                    ([f"{p['name']}F.values", f"{p['name']}F.rows", f"{p['name']}F.cols"]`	matrix の引数なら、flatten した結果の values・rows・cols の 3 つ…
218	`                     if p["js"] == "matrix" else [p["name"]])],`	…そうでなければ引数名そのまま。
219	`        "result": result,`	戻り値の情報。
220	`        "imports": imports,`	必要な import の集合。
221	`    }`	辞書の終わり。
222		
223		

⑨ 公開名を作る（camel_word / base_export_name / resolve_name_collisions）

JS から呼ぶときの名前を、クラス名とメソッド名から自動で作ります。@JSExport はオーバーロードできないので、重複は With〇〇 を付けて区別します。

行	コード	説明
224	`# ----------------------------------------------------------------------------`	区切り線（ここから公開名の作り方）。
225	`# 公開名（JS から呼ぶときの名前）`	JS から呼ぶときの名前を作るまとまり。
226	`# ----------------------------------------------------------------------------`	区切り線。
227		
228	`def camel_word(word):`	大文字が続く略語を 1 語にする関数。
229	`    """FFT → Fft、QZ → Qz のように、大文字の連続を 1 語として扱う。"""`	関数の説明。FFT → Fft、QZ → Qz。
230	`    return re.sub(r"[A-Z]{2,}(?=[A-Z][a-z]|$)", lambda m: m.group(0).capitalize(), word)`	正規表現で「大文字 2 文字以上の連続（直後が大文字＋小文字か、文字列の終わり）」を探し、capitalize（先頭だけ大文字）に置き換える。
231		
232		
233	`def base_export_name(class_name, method_name):`	公開名の基本形を作る関数。

行	コード	説明
234	`    """DoubleRealSchurDecomposer.decompose → realSchurDecompose のように名前を作る。"""`	関数の説明と例。
235	`    prefix = next(v for k, v in ENTRY_PREFIXES.items() if class_name.startswith(k))`	クラス名の先頭（DoubleReal / DoubleComplex）に対応する語（real / complex）を探す。next は条件に合う最初の 1 つを取り出す。
236	`    core = class_name[len(next(k for k in ENTRY_PREFIXES if class_name.startswith(k))):]`	クラス名から先頭部分を取り除いた残り（例：SchurDecomposer）を core とする。
237	`    for suffix in ENTRY_SUFFIXES:`	末尾のルール（Solver など）を順に調べる。
238	`        if core.endswith(suffix):`	core がその末尾で終わっていれば…
239	`            core = core[: -len(suffix)]`	末尾を取り除く（SchurDecomposer → Schur）。[: -len(suffix)] は「後ろから len 文字を除いた部分」。
240	`            break`	1 つ取り除いたらループを抜ける。
241	`    core = camel_word(CORE_ABBREVIATIONS.get(core, core))`	言い換え表にあれば言い換え（SingularValue → Svd）、その後略語を整える（FFT → Fft）。
242	`    # メソッド名がクラスの中心部分で始まるなら重ねない（FFT.fft → realFft、QZ.qzDecompose → realQzDecompose）`	下の if の説明。
243	`    if core and method_name.lower().startswith(core.lower()):`	メソッド名が core で始まっていれば（例：FFT の fft、QZ の qzDecompose）…
244	`        core = ""`	core を重ねて書かないよう空にする（realFftFft にならず realFft になる）。
245	`    return prefix + core + method_name[0].upper() + method_name[1:]`	real ＋ Schur ＋ メソッド名の先頭を大文字にしたもの（Decompose）をつなげて返す。
246		
247		
248	`def resolve_name_collisions(methods):`	公開名の重複を解決する関数。
249	`    """オーバーロードなどで公開名が重なったら、引数の多いほうに With〇〇 を付けて区別する。"""`	関数の説明。オーバーロードで同じ名前になったものに With〇〇 を付ける。
250	`    groups = {}`	「公開名 → その名前のメソッドのリスト」の辞書。
251	`    for m in methods:`	全メソッドについて…
252	`        groups.setdefault(m["export_name"], []).append(m)`	公開名ごとにグループ分けする。setdefault は「キーが無ければ空リストを入れてから、その値を返す」。
253	`    for group in groups.values():`	グループごとに…

行	コード	説明
254	`        if len(group) == 1:`	1 つしか無ければ重複していないので…
255	`            continue`	何もせず次へ。
256	`        group.sort(key=lambda m: len(m["params"]))`	引数の少ない順に並べる。lambda は名前の無い小さな関数で、並べ替えの基準（引数の数）を返す。
257	`        base_params = {p["name"] for p in group[0]["params"]}`	一番引数の少ないメソッドの引数名の集合。
258	`        for m in group[1:]:`	2 番目以降のメソッドについて…
259	`            extra = [p["name"] for p in m["params"] if p["name"] not in base_par ams]`	基準より多い引数（追加の引数）の名前を集める。
260	`            m["export_name"] += "With" + "".join(n[0].upper() + n[1:] for n in e xtra)`	名前の後ろに With ＋ 追加の引数名を付ける（例：complexExp → complexExpWithTolerance）。
261	`    names = [m["export_name"] for m in methods]`	全メソッドの公開名のリスト。
262	`    duplicates = {n for n in names if names.count(n) > 1}`	2 回以上出てくる名前の集合（重複が残っていないかの最終確認）。
263	`    if duplicates:`	それでも重複があれば…
264	`        raise SystemExit(f"公開名が重複しています: {sorted(duplicates)}")`	エラーメッセージを出してプログラムを止める（重複した名前のまま生成すると Java がコンパイルできないため）。
265		
266		

⑩ build_bridges：全体を組み立てる

⑥〜⑨ を組み合わせ、nfc のパッケージごとのブリッジ（EigWasm など）にまとめます。

行	コード	説明
267	`# ----------------------------------------------------------------------------`	区切り線（ここから全体のまとめ）。
268	`# まとめ：テンプレートに渡すデータを作る`	各関数を組み合わせて、テンプレートに渡すデータを作るまとまり。
269	`# ----------------------------------------------------------------------------`	区切り線。
270		
271	`def build_bridges(classes):`	全体を組み立てる関数。classes は JSON の全クラス。
272	`    """入口クラスのメソッドを変換し、nfc のパッケージごとにまとめる。"""`	関数の説明。

行	コード	説明
273	`    class_by_name = {c["name"]: c for c in classes}`	「クラス名 → クラス」の辞書を作る（辞書内包表記）。箱の判定や親クラスの検索で、名前からすぐ引けるようにするため。
274	`    bridges = {}   # パッケージ名（eig など）→ ブリッジ 1 つ分`	nfc のパッケージ名（eig など）→ ブリッジ 1 つ分の情報、の辞書。
275	`    skipped = []`	対象外にしたメソッドを記録するリスト。
276		
277	`    for entry in select_entry_classes(classes):`	選ばれた入口クラスを 1 つずつ処理する。
278	`        package = entry["package"].split(".")[-1]`	パッケージ名の最後の部分を取り出す（org.mklab.nfc.eig → eig）。
279	`        bridge = bridges.setdefault(package, {`	そのパッケージのブリッジが無ければ作り、あればそれを取り出す。
280	`            "package": entry["package"],`	元のパッケージ名（コメント用）。
281	`            "class_name": package.capitalize() + "Wasm",`	ブリッジのクラス名（eig → EigWasm）。capitalize で先頭を大文字にする。
282	`            "methods": [],`	このブリッジに入れるメソッドのリスト。
283	`            "imports": set(),`	このブリッジで必要な import の集合。
284	`        })`	辞書と setdefault の終わり。
285	`        for method in entry["methods"]:`	入口クラスのメソッドを 1 つずつ処理する。
286	`            try:`	try：この中で Skip 例外が起きたら except に飛ぶ。
287	`                m = build_method(entry, method, class_by_name)`	手順 C・D をまとめて行う build_method を呼ぶ。
288	`            except Skip as reason:`	Skip が投げられたら、その理由を reason として受け取る。
289	`                skipped.append({"source": f"{entry['name']}.{method['name']}", "reason": str(reason)})`	対象外リストに「どのメソッドを、なぜ」を記録する。
290	`                continue`	このメソッドは飛ばして次へ。
291	`            m["bridge"] = bridge["class_name"]`	どのブリッジに入るかを記録する。
292	`            bridge["methods"].append(m)`	ブリッジのメソッドリストに追加する。
293	`            bridge["imports"] |= m["imports"]`	必要な import をブリッジの import 集合に足す（|= は集合の和）。
294		
295	`    bridges = [b for b in bridges.values() if b["methods"]]`	メソッドが 1 つも無いブリッジは除く。
296	`    resolve_name_collisions([m for b in bridges for m in b["methods"]])`	全ブリッジの全メソッドを 1 列にして、公開名の重複を解決する。
297	`    for b in bridges:`	各ブリッジについて…

行	コード	説明
298	`        b["imports"] = sorted(b["imports"])`	import の集合を並べ替えたリストにする（毎回同じ順番で出力するため）。
299	`    return sorted(bridges, key=lambda b: b["class_name"]), skipped`	クラス名順に並べたブリッジのリストと、対象外リストを返す。
300		
301		

⑪ render_all：テンプレートでファイルを書き出す

jinja2 の基本の流れ（Environment → get_template → render → 書き込み）です。参考記事の後半とほぼ同じ形です。

行	コード	説明
302	`def render_all(bridges, skipped, json_path):`	テンプレートを使ってファイルを書き出す関数。
303	`    env = Environment(`	jinja2 の環境（設定）を作る。
304	`        loader=FileSystemLoader(BASE_DIR / "templates"),`	テンプレートは Python/templates/ フォルダから読む。
305	`        trim_blocks=True,          # {% %} の直後の改行を消す`	trim_blocks：{% %} の行の直後の改行を消す（生成コードに余計な空行が入らない）。
306	`        lstrip_blocks=True,        # {% %} の前のインデントを消す`	lstrip_blocks：{% %} の前のインデントを消す（テンプレートでは字下げして読みやすく書ける）。
307	`        keep_trailing_newline=True,`	ファイル末尾の改行を残す。
308	`    )`	Environment の設定の終わり。
309	`    common = {"package": JAVA_PACKAGE, "bridges": bridges, "skipped": skipped, "json_path": json_path}`	どのテンプレートにも共通で渡すデータ。この辞書のキーが、テンプレート内の変数名になる。
310		
311	`    JAVA_OUT_DIR.mkdir(parents=True, exist_ok=True)`	Java の出力先フォルダを作る（親フォルダごと、既にあってもエラーにしない）。
312	`    outputs = {JAVA_OUT_DIR / "Matrices.java": env.get_template("matrices.java.j2").render(common),`	「出力先 → 生成した文字列」の辞書を作る。Matrices.java は matrices.java.j2 から。
313	`                JAVA_OUT_DIR / "NfcWasm.java": env.get_template("nfc_wasm.java.j2").render(common),`	NfcWasm.java は nfc_wasm.java.j2 から。
314	`                JS_OUT_FILE: env.get_template("nfc.js.j2").render(common),`	JS ラッパーは nfc.js.j2 から。
315	`                REPORT_FILE: env.get_template("report.md.j2").render(common)}`	レポートは report.md.j2 から。
316	`    for bridge in bridges:`	ブリッジはパッケージごとに 1 ファイルなので、ループで作る。

行	コード	説明
317	`        outputs[JAVA_OUT_DIR / f"{bridge['class_name']}.java"] = \`	出力先は EigWasm.java など。行末の \ は「次の行に続く」という意味。
318	`            env.get_template("bridge.java.j2").render(common, bridge=bridge)`	共通データに加えて、bridge=そのブリッジ を渡して render する。
319		
320	`    for path, text in outputs.items():`	作った文字列を順にファイルへ書き出す。
321	`        path.parent.mkdir(parents=True, exist_ok=True)`	出力先の親フォルダが無ければ作る（build/ や webapp/ 用）。
322	`        path.write_text(text, encoding="utf-8")`	UTF-8 でファイルに書き込む。
323	`        print(f"生成: {path.relative_to(PROJECT_ROOT)}")`	書き出したファイルを、TeaVM フォルダからの相対パスで表示する。
324		
325		

⑫ main：プログラムの入口

引数を受け取り、JSON を読み、⑩ と ⑪ を呼びます。

行	コード	説明
326	`def main():`	プログラムの入口となる関数。
327	`    parser = argparse.ArgumentParser(description="nfc_api.json から TeaVM 用のブリッジコードを生成する")`	コマンドライン引数の解析器を作る。
328	`    parser.add_argument("--json", type=Path, default=DEFAULT_JSON, help="JavaParser が出力した JSON")`	--json オプションを定義。省略すると build/nfc_api.json を使う。type=Path で受け取った文字列を Path に変換する。
329	`    args = parser.parse_args()`	実際に引数を解析する。args.json で値を取り出せる。
330		
331	`    with open(args.json, encoding="utf-8") as f:`	JSON ファイルを UTF-8 で開く。with を使うと、使い終わったら自動で閉じる。
332	`        api = json.load(f)`	JSON を読み込んで Python の辞書に変換する。
333		
334	`    bridges, skipped = build_bridges(api["classes"])`	テンプレートに渡すデータを作る（手順 A〜D と公開名の決定）。
335	`    render_all(bridges, skipped, args.json)`	テンプレートでファイルを生成する。
336		
337	`    count = sum(len(b["methods"]) for b in bridges)`	全ブリッジのメソッド数を合計する。

行	コード	説明
338	`    print(f"公開したメソッド: {count} 個 / 対象外: {len(skipped)} 個（理由は {REPORT_FILE.relative_to(PROJECT_ROOT)}）")`	結果の概要を表示する。
339		
340		
341	`if __name__ == "__main__":`	このファイルを直接実行したときだけ main() を呼ぶ（import されたときは呼ばない）、Python の決まり文句。
342	`    main()`	main 関数を実行する。

テンプレート（templates/）の 1 行ずつの説明

テンプレートの中で、**{{ }}**・**{% %}**・**{# #}** の部分が jinja2 の構文です（色付きで表示）。それ以外の部分は、そのまま出力される Java / JS / Markdown です。

ブリッジ（EigWasm.java など）のテンプレート（bridge.java.j2）

nfc のパッケージ 1 つ分のブリッジを作ります。各メソッドは「引数の変換 → nfc の呼び出し → pack」の 3 段です。変換の中身は Python 側で決まっているので、テンプレートは並べるだけです。

行	コード	説明
1	`{# nfc のパッケージ 1 つ分のブリッジ。各メソッドは「引数の変換 → 呼び出し → pack」の 3 段 #}`	{# … #} は jinja2 のコメント。生成されるファイルには出力されない。
2	`package {{ package }};`	{{ package }} に render で渡した値（org.example.generated）が入る。
3		
4	`{% for imp in bridge.imports %}`	{% for %} で bridge.imports（import する完全修飾名のリスト）を 1 つずつ imp として取り出す。
5	`import {{ imp }};`	import 文を 1 行出力する。例：import org.mklab.nfc.eig.DoubleRealSchurDecomposer;
6	`{% endfor %}`	for の終わり。
7		
8	`/**`	Javadoc の始まり。
9	` * {@code {{ bridge.package }}} と JS をつなぐクラスです（JavaGenerator.py が自動生成。手で編集しないこと）。`	元のパッケージ名を埋め込んだ説明。自動生成なので手で編集しないよう注意書きを入れている。
10	` */`	Javadoc の終わり。
11	`final class {{ bridge.class_name }} {`	ブリッジのクラス宣言。クラス名（EigWasm など）を埋め込む。package-private（public なし）なので同じパッケージの NfcWasm からだけ使える。
12		
13	`    private {{ bridge.class_name }}() {`	private コンストラクタ。static メソッドだけのクラスなので new させない。
14	`    }`	コンストラクタの終わり。
15	`{% for m in bridge.methods %}`	このブリッジのメソッドを 1 つずつ m として取り出す。
16		
17	`    /** {@code {{ m.source }}} → {{ m.original_type }} */`	元のメソッド名と元の戻り値の型をコメントとして出力する。

行	コード	説明
18	`    static {{ m.result.java }} {{ m.export_name }}({{ m.java_params | join(", ") }}) {`	メソッド宣言。戻り値の型・公開名・引数（java_params を「, 」でつなげたもの）を埋め込む。| join は jinja2 のフィルタ。
19	`{% if m.throws %}`	throws がある（例外を投げるかもしれない）メソッドなら…
20	`        try {`	try { を出力する。
21	`{% endif %}`	if の終わり。
22	`{% set indent = "            " if m.throws else "        " %}`	{% set %} でテンプレート内の変数を作る。try の中なら字下げを 1 段深く（スペース 12 個）、そうでなければ 8 個。
23	`{% if m.result.expr %}`	戻り値を返すメソッド（void 以外）なら…
24	`{{ indent }}{{ m.original_type }} result = {{ m.call }};`	「元の型 result = nfc の呼び出し;」を出力する。型を明示して書く（var は使わない）。
25	`{{ indent }}return {{ m.result.expr }};`	「return 変換式;」を出力する。例：return Matrices.pack(result.getU(), result.getT());
26	`{% else %}`	void のメソッドなら…
27	`{{ indent }}{{ m.call }};`	呼び出しだけを出力する。
28	`{% endif %}`	if の終わり。
29	`{% if m.throws %}`	throws があるメソッドなら…
30	`        } catch (Exception e) {`	例外を捕まえる。
31	`            throw new RuntimeException(e);`	RuntimeException に包んで投げ直す（検査例外を JS 側にそのまま伝えるため）。
32	`        }`	catch の終わり。
33	`{% endif %}`	if の終わり。
34	`    }`	メソッドの終わりの } を出力する。
35	`{% endfor %}`	for の終わり。
36	`}`	クラスの終わりの } を出力する。

公開窓口（NfcWasm.java）のテンプレート（nfc_wasm.java.j2）

@JSExport を付けたメソッドを並べ、ブリッジに処理を渡すだけのクラスを作ります。TeaVM の mainClass にこのクラスを指定します。

行	コード	説明
1	`{# 公開窓口。@JSExport を付けて、ブリッジに処理を渡すだけ #}`	jinja2 のコメント。
2	`package {{ package }};`	パッケージ名を埋め込む。
3		
4	`import org.teavm.jso.JSExport;`	TeaVM の @JSExport アノテーションを import する。
5		
6	`/**`	Javadoc の始まり。
7	`* 自動生成したブリッジを JS に公開する入口です（JavaGenerator.py が自動生成。手で編集しないこと）。`	このクラスの説明。
8	`*`	Javadoc の空行。
9	`* <p>TeaVM の mainClass にこのクラスを指定すると、下の @JSExport メソッドが JS から呼べるようになります。</p>`	TeaVM の mainClass にこのクラスを指定すると、@JSExport メソッドが JS から呼べるようになることを説明している。
10	`*/`	Javadoc の終わり。
11	`public final class NfcWasm {`	公開窓口のクラス。TeaVM が読むので public にする。
12		
13	`    private NfcWasm() {`	private コンストラクタ（new させない）。
14	`    }`	コンストラクタの終わり。
15		
16	`    public static void main(String[] args) {`	TeaVM の mainClass には main メソッドが必要なので用意する。
17	`        // JS から exports を呼ぶだけなので何もしない`	中身は空。JS からは exports 経由で各メソッドを呼ぶだけ。
18	`    }`	main の終わり。
19	`{% for b in bridges %}`	ブリッジ（パッケージ）ごとに繰り返す。
20		
21	`    // ---- {{ b.package }} ----`	どのパッケージのメソッドかが分かる区切りコメントを出力する。
22	`{% for m in b.methods %}`	そのブリッジのメソッドごとに繰り返す。
23		

行	コード	説明
24	`    @JSExport`	@JSExport を付ける。これで JS から呼べるメソッドになる。
25	`    public static {{ m.result.java }} {{ m.export_name }}({{ m.java_params | join(", ") }}) {`	メソッド宣言。ブリッジと同じ戻り値・名前・引数にする。
26	`        {{ "return " if m.result.expr }}{{ b.class_name }}.{{ m.export_name }}({{ m.java_args | join(", ") }});`	ブリッジの同名メソッドに引数をそのまま渡す。void でなければ先頭に return を付ける（{{ "return " if … }} は条件付きの出力）。
27	`    }`	メソッドの終わり。
28	`{% endfor %}`	内側の for の終わり。
29	`{% endfor %}`	外側の for の終わり。
30	`}`	クラスの終わり。

JS ラッパー（nfc.generated.js）のテンプレート（nfc.js.j2）

前半（8〜49 行）は毎回同じ内容の補助関数です。後半（51 行〜）で、各メソッドについて「flatten して Wasm を呼び、戻り値を unpack する」コードを作ります。

行	コード	説明
1	`{# JS 側のラッパー。2 次元配列を flatten して渡し、戻ってきた配列を unpack して返す #}`	jinja2 のコメント（出力されない）。
2	`// NFC (TeaVM WasmGC) を 2 次元配列で扱うためのラッパー（JavaGenerator.py が自動生成。手で編集しないこと）。`	JS のコメント。自動生成であることの注意書き。
3	`// mainClass を {{ package }}.NfcWasm にしてビルドした wasm と組み合わせて使う。`	mainClass を生成版の NfcWasm にしてビルドした wasm と組み合わせて使うことを書いている。
4	`//`	空のコメント行。
5	`//   const nfc = await loadNfcGenerated("wasm-gc/nfc.wasm");`	使い方の例：wasm を読み込んでラッパーを受け取る。
6	`//   const { U, T } = nfc.realSchurDecompose([[1, 2], [3, 4]]);`	使い方の例：2 次元配列を渡すと { U, T } が返る。
7		
8	`async function loadNfcGenerated(wasmUrl = "wasm-gc/nfc.wasm") {`	ラッパーを作る async 関数。他の JS と名前がぶつからないよう、補助関数はすべてこの中に入れている。
9	`  /** 2次元配列 → { values: 行優先の1次元配列, rows, cols } */`	flatten の説明。
10	`  function flatten(matrix) {`	2 次元配列を「1 次元配列 + 行数 + 列数」に変換する関数。
11	`    const rows = matrix.length;`	行数。

行	コード	説明
12	`  const cols = rows === 0 ? 0 : matrix[0].length;`	列数（行が 0 なら 0）。
13	`  const values = new Array(rows * cols);`	要素を入れる 1 次元配列を用意する。
14	`  for (let r = 0; r < rows; r++) {`	各行について…
15	`    if (matrix[r].length !== cols) throw new Error("行ごとの列数が揃っていません");`	列数が揃っていなければエラーにする。
16	`    for (let c = 0; c < cols; c++) values[r * cols + c] = matrix[r][c];`	行優先（1 行目の全要素、2 行目の全要素…）の順に詰める。
17	`  }`	for の終わり。
18	`  return { values, rows, cols };`	values・rows・cols をまとめたオブジェクトを返す。
19	`}`	flatten の終わり。
20		
21	`/** [k, r1, c1, ..., data...] → 2次元配列の配列 */`	unpack の説明。Java の Matrices.pack の逆。
22	`function unpack(packed) {`	pack された 1 本の配列を、2 次元配列の配列に戻す関数。
23	`  const count = packed[0];`	先頭の数字は行列の個数。
24	`  let offset = 1 + count * 2;`	要素データが始まる位置（個数 1 つ + 行数・列数のペア × 個数 の後ろ）。
25	`  const result = [];`	結果を入れる配列。
26	`  for (let i = 0; i < count; i++) {`	行列の個数だけ繰り返す。
27	`    const rows = packed[1 + i * 2];`	i 番目の行列の行数。
28	`    const cols = packed[2 + i * 2];`	i 番目の行列の列数。
29	`    const m = [];`	1 つの行列を組み立てる配列。
30	`    for (let r = 0; r < rows; r++) {`	行ごとに…
31	`      m.push(Array.from(packed.slice(offset, offset + cols)));`	列数分の要素を切り出して 1 行にする。
32	`      offset += cols;`	読む位置を進める。
33	`    }`	for の終わり。
34	`    result.push(m);`	できた行列を結果に追加する。
35	`  }`	for の終わり。
36	`  return result;`	行列の配列を返す。

行	コード	説明
37	`  }`	unpack の終わり。
38		
39	`  /** unpack した行列の並びに名前を付ける。kinds の値が "vector" なら 1 x n 行列を 1 次元配列に戻す */`	named の説明。
40	`  function named(matrices, kinds) {`	unpack した行列の並びに名前を付けて、{ U: …, T: … } のようなオブジェクトにする関数。
41	`    const out = {};`	結果のオブジェクト。
42	`    Object.keys(kinds).forEach((key, i) => {`	kinds のキー（U, T など）を順番に取り出し、i 番目の行列と対応させる。
43	`      out[key] = kinds[key] === "vector" ? matrices[i][0] : matrices[i];`	"vector" なら 1 x n 行列の 1 行目だけ（1 次元配列）、それ以外は行列のまま入れる。
44	`    });`	forEach の終わり。
45	`    return out;`	オブジェクトを返す。
46	`  }`	named の終わり。
47		
48	`  const teavm = await TeaVM.wasmGC.load(wasmUrl);`	TeaVM のランタイム（nfc.wasm-runtime.js）で wasm を読み込む。
49	`  const ex = teavm.exports;`	@JSExport したメソッドが入っている exports を取り出す。
50		
51	`  return {`	ラッパーのオブジェクトを返す。
52	`    exports: ex,`	生の exports にもアクセスできるようにしておく。
53	`{% for b in bridges %}`	ブリッジ（パッケージ）ごとに繰り返す。
54		
55	`    // ---- {{ b.package }} ----`	パッケージ名の区切りコメント。
56	`{% for m in b.methods %}`	メソッドごとに繰り返す。
57	`    /** {{ m.source }} */`	元のメソッド名をコメントとして出力する。
58	`    {{ m.export_name }}({{ m.params | map(attribute="name") | join(", ") }}) {`	ラッパーのメソッド宣言。引数は元の引数名だけ（map(attribute="name") で各引数の name を取り出して join）。
59	`{% for p in m.params if p.js == "matrix" %}`	{% for … if … %} で、matrix の引数だけを取り出して繰り返す。

行	コード	説明
60	`      const {{ p.name }}F = flatten({{ p.name }});`	その引数を flatten した結果を「名前F」という変数に入れる（例：const aF = flatten(a);）。
61	`{% endfor %}`	for の終わり。
62	`{% set call = "ex." ~ m.export_name ~ "(" ~ (m.js_args | join(", ")) ~ ")" %}`	Wasm を呼ぶ式を set で作っておく。~ は jinja2 の文字列連結。例：ex.realSchurDecompose(aF.values, aF.rows, aF.cols)
63	`{% if m.result.js == "none" %}`	戻り値の扱いで分岐する。none（void）なら…
64	`      {{ call }};`	呼ぶだけで何も返さない。
65	`{% elif m.result.js == "plain" %}`	plain（数値・真偽値）なら…
66	`      return {{ call }};`	そのまま返す。
67	`{% elif m.result.js == "vector" %}`	vector（1 次元配列）なら…
68	`      return unpack({{ call }})[0][0];`	unpack して 1 つ目の行列の 1 行目を返す。
69	`{% elif m.result.js == "matrix" %}`	matrix（行列 1 つ）なら…
70	`      return unpack({{ call }})[0];`	unpack して 1 つ目の行列を返す。
71	`{% elif m.result.js == "matrices" %}`	matrices（行列の配列）なら…
72	`      return unpack({{ call }});`	unpack した配列をそのまま返す。
73	`{% elif m.result.js == "object" %}`	object（箱）なら…
74	`      return named(unpack({{ call }}), { {% for f in m.result.fields %}{{ f.key }}: "{{ f.js }}"{{ ", " if not loop.last }}{% endfor %} });`	unpack して named で名前を付けて返す。例：named(unpack(…), { U: "matrix", T: "matrix" })
75	`{% endif %}`	if の終わり。
76	`    },`	メソッドの終わり（オブジェクトの要素なので , を付ける）。
77	`{% endfor %}`	内側の for の終わり。
78	`{% endfor %}`	外側の for の終わり。
79	`  };`	返すオブジェクトの終わり。
80	`}`	関数の終わり。

レポート（codegen_report.md）のテンプレート（report.md.j2）

公開したメソッドと、対象外にしたメソッドの一覧を Markdown の表で出力します。

行	コード	説明
1	`# コード生成レポート`	Markdown の見出し。
2		
3	`入力: `{{ json_path }}` （JavaGenerator.py が自動生成）`	入力に使った JSON のパスを埋め込む。
4		
5	`## 公開したメソッド（{{ bridges | sum(attribute="methods", start=[]) | length }} 個）`	公開したメソッドの数。sum(attribute="methods", start=[]) で全ブリッジのメソッドリストを 1 つにつなげ、length で数える。
6		
7	`| 公開名 | 元のメソッド | 元の戻り値 | JS での戻り値 |`	表の見出し行。
8	`|---|---|---|---|`	Markdown の表の区切り行。
9	`{% for b in bridges %}`	ブリッジごとに…
10	`{% for m in b.methods %}`	メソッドごとに…
11	`| `{{ m.export_name }}` | `{{ m.source }}` | `{{ m.original_type }}` | {% if m.result.js == "object" %}`{ {{ m.result.fields | map(attribute="key") | join(", ") }} }`{% else %}{{ m.result.js }}{% endif %} |`	表の 1 行を出力する。箱なら { U, T } のように中身の名前を、それ以外なら JS での形（matrix など）を書く。
12	`{% endfor %}`	内側の for の終わり。
13	`{% endfor %}`	外側の for の終わり。
14		
15	`## 対象外にしたメソッド（{{ skipped | length }} 個）`	対象外にしたメソッドの数（skipped | length）。
16		
17	`| 元のメソッド | 理由 |`	表の見出し行。
18	`|---|---|`	表の区切り行。
19	`{% for s in skipped %}`	対象外リストを 1 つずつ…
20	`| `{{ s.source }}` | {{ s.reason }} |`	「元のメソッド | 理由」の 1 行を出力する。
21	`{% endfor %}`	for の終わり。